

area where you edit the text is called the main buffer, and the area at the bottom of the Emacs window, where commands are entered, is called the mini buffer.

Start Emacs and to create a new file, press `C-x C-f`. This will prompt you to enter a file name. Let us call our file 'buggyFactorial.cpp'. Once the file is open, type in the code shown below:

```
#include<iostream>
#include <assert.h>
int factorial(int num) {
    int product = 1;
    while(num-->0) {
        product *= num;
    }
    return product;
}
int main() {
    int result = factorial(5);
    assert(result == 120);
}
```

Save the file with `C-x C-s`. Once the file is saved, it's time to compile the code. Press `M-x` and in the prompt that comes up, type in `compile` and hit `Enter`. Then, in the prompt, replace whatever is there with `g++ -g buggyFactorial.cpp` and again hit `Enter`.

This will open up another buffer in Emacs that will show the status of the compile and, hopefully, if the code typed in is correct, you will get a buffer like the one shown in Figure 2.

To hide this compilation status buffer, make sure your cursor is in the compilation buffer (you can do this without the mouse using `C-x o`-this is used to move the cursor from one open buffer to the other), and then press `C-x 0`. The next step is to run the code and see if it works fine. Press `M-!` and in the mini buffer prompt, type `./a.out`.

See the mini buffer that says the assertion is failed. Clearly, something is wrong with the code, because the factorial (5) is 120. So let's debug the code now.

Debugging the code using GUD

Now, since we have the code compiled, it's time to see what is wrong with it. Press `M-x` and in the prompt, enter `gdb`. In the next prompt that appears, write `gdb -i=mi a.out`, which will start GDB in the Emacs buffer and if everything goes

```
12 int main() {
13     int result = factorial(5);
14     assert(result == 120);
15     return 0;
16 }
17
18
- 233 -: buggyFactorial.cpp C++/l company FlyC Abbrev 16 : 1 Bottom
1 *- mode: compilation; default-directory: "/media/kartha/New Volume1/Vineeth_
Projects/OSFY/Gud/" *-
2 Compilation started at Thu Jul 18 07:05:08
3
4 g++ buggyFactorial.cpp -g
5
6 Compilation finished at Thu Jul 18 07:05:09
7
8 % 216 1: *compilation* Compilation:exit [0] company 1 : 0 All
9 Compilation finished
```

Figure 2: Compilation status

```
emacs@n1rv4n4
File Edit Options Buffers Tools C++ Help
1 #include<iostream>
2 #include <assert.h>
3
4 int factorial(int num) {
5     int product = 1;
6     while(num-->0) {
7         product *= num;
8     }
9     return product;
10 }
11
12 int main() {
13     int result = factorial(5);
14     assert(result == 120);
15     return 0;
16 }
17
18
- 233 -: buggyFactorial.cpp C++/l company FlyC Abbrev 16 : 1 All
a.out: buggyFactorial.cpp:14: int main(): Assertion 'result == 120' failed.
```

Figure 3: Output of the code in the mini buffer

```
emacs@n1rv4n4
File Edit Options Buffers Tools Gdb Complete In/Out Signals Help
P P- Run Run Next Line Step Line Up Stack
1 Current directory is /media/kartha/New Volume1/Vineeth_Projects/OSFY/Gud/
2 GNU gdb (Ubuntu 8.1-0ubuntu3) 8.1.0.20180409-git
3 Copyright (C) 2018 Free Software Foundation, Inc.
4 License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
5 This is free software: you are free to change and redistribute it.
6 There is NO WARRANTY, to the extent permitted by law. Type "show copying"
7 and "show warranty" for details.
8 This GDB was configured as "x86_64-linux-gnu".
9 Type "show configuration" for configuration details.
10 For bug reporting instructions, please see:
11 <http://www.gnu.org/software/gdb/bugs/>.
12 Find the GDB manual and other documentation resources online at:
13 <http://www.gnu.org/software/gdb/documentation/>.
14 For help, type "help".
15 Type "apropos word" to search for commands related to "word"...
16 Reading symbols from a.out...done.
17 (gdb)
* 854 1: *gud-a.out* Debugger:run [ready] company 17 : 6 All
Target doesn't support non-stop mode. Turning it off.
```

Figure 4: The GDB buffer in Emacs